

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I



BÁO CÁO CUỐI KỲ
HỌC PHẦN: THỰC TẬP CƠ SỞ
ĐỀ TÀI

Giảng viên hướng dẫn	: TS. Kim Ngọc Bach
Mã sinh viên	: B23DCCN397
Tên sinh viên	: Nguyễn Nhật Huy
Lớp hành chính	: D23CQCN05-B
Nhóm thực tập	: 11

Hà Nội – 2026

I. Giới thiệu chung

1. Đặt vấn đề

- Trong những năm gần đây, sự phát triển mạnh mẽ của trí tuệ nhân tạo, thị giác máy tính, và robot tự hành đã thúc đẩy nhu cầu xây dựng các hệ thống có khả năng nhận thức môi trường xung quanh một cách chính xác và toàn diện. Các ứng dụng như xe tự lái, robot dịch vụ, robot công nghiệp, hệ thống giám sát thông minh hay phương tiện bay không người lái yêu cầu khả năng hiểu được cấu trúc của môi trường xung quanh hoạt động để đưa ra các quyết định phù hợp.

- Để nhận thức môi trường, hệ thống cần xác định được các đối tượng xuất hiện trong không gian, vị trí giữa chúng, mối quan hệ giữa các đối tượng xuất hiện trong không gian, vị trí của chúng, mối quan hệ giữa các đối tượng và các đặc trưng ngữ nghĩa của từng thành phần trong cảnh. Trong nhiều năm qua, dữ liệu hình ảnh RGB được sử dụng rộng rãi cho các bài toán nhận dạng và phân loại đối tượng. Tuy nhiên, dữ liệu ảnh hai chiều thường gặp nhiều hạn chế trong việc mô tả thông tin chiều sâu và cấu trúc hình học của môi trường thực tế.

- Sự xuất hiện của cảm biến LiDAR (Light Detection and Ranging) đã mang lại khả năng thu thập dữ liệu không gian ba chiều với độ chính xác cao. LiDAR hoạt động dựa trên nguyên lý phát tia laser và đo thời gian phản hồi để xác định khoảng cách tới các vật thể trong môi trường. Kết quả thu được là tập hợp các điểm trong không gian ba chiều, thường được gọi là **Point Cloud**. So với dữ liệu ảnh RGB, point cloud cung cấp thông tin hình học trực tiếp về môi trường, ít bị ảnh hưởng bởi điều kiện ánh sáng và có độ chính xác cao trong việc đo khoảng cách.

- Mặc dù mang lại nhiều lợi ích, dữ liệu point cloud cũng đặt ra nhiều thách thức trong quá trình xử lý. Không giống như ảnh số được tổ chức dưới dạng lưới pixel đều đặn, point cloud là dữ liệu không có cấu trúc, phân bố không đồng đều và thường chứa số lượng điểm rất lớn. Điều này làm cho các phương pháp xử lý ảnh truyền thống không thể áp dụng trực tiếp.

- Trong lĩnh vực nhận thức môi trường dựa trên point cloud, nhiều bài toán đã được nghiên cứu như phát hiện đối tượng (Object Detection), phân đoạn ngữ nghĩa (Semantic Segmentation) và phân đoạn thực thể (Instance Segmentation). Tuy nhiên, mỗi bài toán chỉ cung cấp một phần thông tin về cảnh quan.

Semantic Segmentation cho phép xác định loại đối tượng của từng điểm nhưng không thể phân biệt các đối tượng cùng loại. Ngược lại, Instance Segmentation có thể phân biệt từng đối tượng riêng biệt nhưng chưa mô tả đầy đủ thông tin ngữ nghĩa của toàn cảnh.

- Để khắc phục hạn chế trên, bài toán Panoptic Segmentation được đề xuất nhằm kết hợp ưu điểm của Semantic Segmentation và Instance Segmentation. Trong bài toán này, mỗi điểm trong không gian được gán đồng thời nhãn ngữ nghĩa và mã thực thể, cho phép hệ thống vừa nhận biết loại đối tượng vừa phân biệt được từng cá thể riêng biệt trong cùng một lớp.
- Panoptic Segmentation hiện được xem là một trong những hướng nghiên cứu quan trọng trong lĩnh vực nhận thức môi trường cho xe tự hành và robot thông minh. Tuy nhiên, việc áp dụng Panoptic Segmentation cho dữ liệu point cloud vẫn còn nhiều khó khăn do tính chất thưa và không đồng đều của dữ liệu LiDAR.

2. Mục tiêu đề tài

2.1. Mục tiêu tổng quát

- Nghiên cứu và xây dựng hệ thống phân đoạn panoptic 3D trên dữ liệu đám mây điểm LiDAR sử dụng kiến trúc Panoptic-PolarNet nhằm nâng cao khả năng nhận thức môi trường cho các hệ thống tự hành.

2.2. Mục tiêu cụ thể

- Đề tài tập trung thực hiện các mục tiêu sau:
 - Nghiên cứu dữ liệu point cloud và cảm biến LiDAR.
 - Tìm hiểu các phương pháp Semantic Segmentation và Instance Segmentation.
 - Nghiên cứu bài toán Panoptic Segmentation.
 - Tìm hiểu kiến trúc PolarNet và Panoptic-PolarNet.
 - Cài đặt mô hình bằng PyTorch.
 - Huấn luyện mô hình trên bộ dữ liệu SemanticKITTI.
 - Đánh giá hiệu năng bằng các chỉ số PQ, SQ và RQ.
 - Trực quan hóa và phân tích kết quả thực nghiệm.

3. Nội dung nghiên cứu

- Để đạt được các mục tiêu đề ra, đề tài tiến hành nghiên cứu các nội dung chính sau:

Nghiên cứu lý thuyết

- Point Cloud Processing.
- LiDAR Perception.
- Semantic Segmentation.
- Instance Segmentation.

- Panoptic Segmentation.

Nghiên cứu mô hình

- Các mô hình xử lý Semantic Segmentation
- Clustering
- Panoptic-PolarNet.
- Semantic Branch.
- Instance Branch.
- Panoptic Fusion.

Nghiên cứu thực nghiệm

- Bộ dữ liệu SemanticKITTI.
- Tiền xử lý dữ liệu.
- Huấn luyện mô hình.
- Đánh giá kết quả.

4. Phạm vi nghiên cứu

- Đề tài tập trung vào:

- Dữ liệu LiDAR 3D.
- Bộ dữ liệu SemanticKITTI.
- Kiến trúc Panoptic-PolarNet.
- Framework PyTorch.

II. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

1 Tổng quan về dữ liệu đám mây điểm (Point Cloud)

1.1 Khái niệm Point Cloud

- Point Cloud là tập hợp các điểm rời rạc trong không gian ba chiều dùng để biểu diễn hình dạng và cấu trúc của các vật thể hoặc môi trường xung quanh. Mỗi điểm trong đám mây điểm thường được xác định bởi tọa độ không gian và một số thuộc tính bổ sung như cường độ phản xạ, màu sắc hoặc thông tin thời gian. Một điểm trong không gian có thể được biểu diễn dưới dạng:

$$P_i = (x_i, y_i, z_i)$$

hoặc

$$P_i = (x_i, y_i, z_i, I_i)$$

Trong đó:

- x_i : tọa độ theo trục X.
- y_i : tọa độ theo trục Y.
- z_i : tọa độ theo trục Z.
- I_i : cường độ phản xạ của tia laser.

- Toàn bộ Point Cloud được biểu diễn như sau:

$$P = P_1, P_2, \dots, P_n$$

với (n) là tổng số điểm trong frame (một lần quét).

- Ngày nay Point Cloud được sử dụng rộng rãi trong nhiều lĩnh vực:

- Xe tự hành.
- Robot tự hành.
- UAV.
- Bản đồ địa hình.
- Khảo sát công trình.
- Mô hình hóa thành phố 3D.
- Thực tế ảo và thực tế tăng cường.

- Đối với bài toán nhận thức môi trường, Point Cloud là nguồn dữ liệu quan trọng giúp hệ thống hiểu được cấu trúc hình học của thế giới thực.

1.2 Các nguồn tạo Point Cloud

Có nhiều phương pháp khác nhau để tạo ra dữ liệu Point Cloud.

- LiDAR: LiDAR là phương pháp phổ biến nhất trong lĩnh vực xe tự hành. Cảm biến phát ra các tia laser và đo thời gian phản hồi để tính khoảng cách tới vật thể.

Ưu điểm:

- Độ chính xác cao.
- Hoạt động tốt trong điều kiện ánh sáng yếu.

- Thu được dữ liệu khoảng cách trực tiếp.

Nhược điểm:

- Chi phí cao.
- Dữ liệu thường thừa ở khoảng cách xa.

- Stereo Camera: Sử dụng hai camera để tái tạo chiều sâu dựa trên thị sai.

Ưu điểm:

- Chi phí thấp.
- Thu được đồng thời ảnh RGB.

Nhược điểm:

- Độ chính xác thấp hơn LiDAR.
- Phụ thuộc nhiều vào điều kiện ánh sáng.

- RGB-D Camera: Cung cấp đồng thời ảnh màu và bản đồ chiều sâu.

Ưu điểm:

- Dễ triển khai.
- Chi phí thấp.

Nhược điểm:

- Tầm hoạt động hạn chế.

1.3 Đặc điểm của Point Cloud

- Khác với ảnh số thông thường, dữ liệu Point Cloud có nhiều đặc điểm riêng biệt.

- Không có cấu trúc lưới
- Ảnh RGB được tổ chức dưới dạng ma trận pixel: $H \times W$
- Trong khi đó Point Cloud chỉ là tập hợp các điểm rời rạc không có cấu trúc cố định.

- Điều này khiến các mạng CNN truyền thống khó áp dụng trực tiếp.

- Mật độ không đồng đều
- Các điểm gần cảm biến thường có mật độ cao hơn các điểm ở xa.

- Điều này dẫn đến hiện tượng phân bố dữ liệu không đồng nhất trong không gian.
 - Dữ liệu thừa
- Một cảnh LiDAR chỉ chứa một số lượng điểm hữu hạn.
- Do đó tồn tại nhiều vùng không gian không có dữ liệu.
- Kích thước lớn
 - Một khung dữ liệu LiDAR có thể chứa hàng trăm nghìn điểm.
 - Việc xử lý trực tiếp đòi hỏi tài nguyên tính toán lớn.

1.4 Thách thức trong xử lý Point Cloud

- Các đặc điểm trên tạo ra nhiều thách thức:
- Khó áp dụng CNN trực tiếp.
 - Chi phí tính toán cao.
 - Mất cân bằng dữ liệu.
 - Nhiều cảm biến.
 - Hiện tượng che khuất (Occlusion).
- Do đó cần xây dựng các phương pháp biểu diễn dữ liệu phù hợp trước khi đưa vào mạng học sâu.

2 Công nghệ LiDAR

2.1 Giới thiệu LiDAR

- LiDAR (Light Detection and Ranging) là công nghệ đo khoảng cách bằng tia laser.
- Nguyên lý hoạt động dựa trên việc phát một chùm tia laser tới vật thể và đo thời gian phản hồi của tia sau khi phản xạ trở lại cảm biến.
- Khoảng cách được tính theo công thức: $d = \frac{ct}{2}$

Trong đó:

- d: khoảng cách.
- c: vận tốc ánh sáng.

- t: thời gian phản hồi.
- LiDAR hiện là cảm biến chủ đạo trong các hệ thống xe tự hành do khả năng tạo ra bản đồ không gian ba chiều chính xác.

2.2 Cấu tạo của hệ thống LiDAR

- Một hệ thống LiDAR thông thường gồm:
- Bộ phát laser: Phát các tia laser tới môi trường.
 - Bộ thu tín hiệu: Nhận tín hiệu phản xạ từ vật thể.
 - Bộ đo thời gian: Xác định thời gian truyền và nhận tín hiệu.
 - Bộ xử lý: Chuyển đổi dữ liệu khoảng cách thành Point Cloud.

2.3 Velodyne HDL-64E

- SemanticKITTI sử dụng dữ liệu từ cảm biến Velodyne HDL-64E.

Thông số chính:

Thông số	Giá trị
Số tia laser	64
Tần số quét	10 Hz
Góc quét ngang	360°
Tầm hoạt động	120 m

- Velodyne HDL-64E là một trong những cảm biến LiDAR phổ biến nhất trong nghiên cứu xe tự hành.

2.4 Ưu điểm của LiDAR

- Đo khoảng cách chính xác.
- Không phụ thuộc ánh sáng môi trường.
- Hoạt động cả ban ngày và ban đêm.
- Tạo bản đồ 3D trực tiếp.

2.5 Nhược điểm của LiDAR

- Giá thành cao.
- Dữ liệu thưa ở khoảng cách xa.

- Dễ bị ảnh hưởng bởi mưa lớn hoặc sương mù.

3. Deep Learning cho dữ liệu Point Cloud

3.1 Tổng quan Deep Learning

- Deep Learning là nhánh của Machine Learning sử dụng mạng nơ-ron nhiều lớp để học đặc trưng từ dữ liệu.
- Trong lĩnh vực Computer Vision, Deep Learning đã đạt được thành công vượt trội trong:
 - Classification.
 - Detection.
 - Segmentation.
 - Tracking.
- Tuy nhiên Point Cloud có cấu trúc khác biệt so với ảnh nên cần các kiến trúc chuyên biệt.

3.2 Các hướng xử lý Point Cloud

- Hiện nay có ba hướng tiếp cận chính.

*** Point-based**

Xử lý trực tiếp từng điểm.

Ví dụ:

- PointNet
- PointNet++
- Point Transformer

Ưu điểm:

- Giữ nguyên dữ liệu gốc.

Nhược điểm:

- Chi phí tính toán lớn.

*** Voxel-based**

- Chia không gian thành các voxel.

Ví dụ:

- VoxelNet
- SECOND
- Cylinder3D

Ưu điểm:

- Dễ áp dụng CNN.

Nhược điểm:

- Mất thông tin khi lượng tử hóa.

* **Projection-based**

- Chiếu Point Cloud sang dạng ảnh.

Ví dụ:

- Range View
- Bird-Eye-View
- Polar Grid

Ưu điểm:

- Tốc độ cao.

Nhược điểm:

- Có thể làm mất một phần thông tin hình học.

Panoptic-PolarNet thuộc nhóm Projection-based.

4. Semantic Segmentation

4.1 Khái niệm

- Semantic Segmentation là bài toán gán nhãn ngữ nghĩa cho từng điểm hoặc từng pixel trong dữ liệu.

- Mục tiêu là xác định đối tượng mà mỗi điểm thuộc về.

Ví dụ:

- Road.
- Building.
- Car.
- Vegetation.

- Đầu ra của Semantic Segmentation là bản đồ nhãn ngữ nghĩa.

4.2 Quy trình Semantic Segmentation

- Quá trình thực hiện gồm:

Bước 1: Trích xuất đặc trưng.

Bước 2: Phân loại từng điểm.

Bước 3: Tạo bản đồ nhãn.

4.3 Các mô hình Semantic Segmentation phổ biến

* Fully Convolutional Network (FCN)

- FCN là kiến trúc đầu tiên thay thế các lớp Fully Connected bằng Convolution.

Ưu điểm:

- Đầu vào kích thước linh hoạt.
- Dự đoán theo từng pixel.

* U-Net

- U-Net sử dụng cấu trúc Encoder–Decoder cùng Skip Connection.

Ưu điểm:

- Bảo toàn thông tin chi tiết.
- Hiệu quả trên dữ liệu y tế.

4.4 Semantic Segmentation cho Point Cloud

Một số mô hình nổi bật:

- SalsaNext
- PolarNet.
- Cylinder3D

Trong đó PolarNet là nền tảng được sử dụng để xây dựng Panoptic-PolarNet.

4.5 Hạn chế của Semantic Segmentation

- Semantic Segmentation chỉ cung cấp thông tin lớp ngữ nghĩa.

Ví dụ:

Ba chiếc xe khác nhau đều được gán nhãn "Car".

- Do đó hệ thống không thể biết được có bao nhiêu đối tượng thực sự xuất hiện trong cảnh.
- Hạn chế này là động lực thúc đẩy sự ra đời của bài toán Instance Segmentation và Panoptic Segmentation.

III. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

1 Tổng quan hệ thống

1.1 Bài toán đặt ra

- Mục tiêu của hệ thống là thực hiện Panoptic Segmentation trên dữ liệu LiDAR 3D.

Đầu vào:

- Point Cloud từ cảm biến LiDAR.

Đầu ra:

- Semantic Label.
- Instance ID.

Ví dụ:

Point	Semantic	Instance
P1	Car	1
P2	Car	1
P3	Car	2
P4	Road	0

- Khác với Semantic Segmentation thông thường, hệ thống cần vừa nhận diện loại đối tượng vừa phân biệt từng thực thể riêng biệt.

1.2 Kiến trúc tổng thể

- Pipeline xử lý của Panoptic-PolarNet:

Hệ thống gồm 5 giai đoạn:

1. Tiền xử lý dữ liệu.
2. Polar Representation.
3. Semantic Prediction.
4. Instance Prediction.
5. Panoptic Fusion.

2 Phân tích dữ liệu đầu vào

2.1 Định dạng dữ liệu SemanticKITTI

- Mỗi frame LiDAR được lưu dưới dạng: 000000.bin
- Mỗi điểm gồm: (x, y, z, intensity)

Ví dụ: (1.25, 3.67, -1.45, 0.72)

2.2 Ground Truth

- SemanticKITTI cung cấp: 000000.label

Trong đó:

Label = Semantic ID + Instance ID

Ví dụ:

Car #1

Car #2

Road

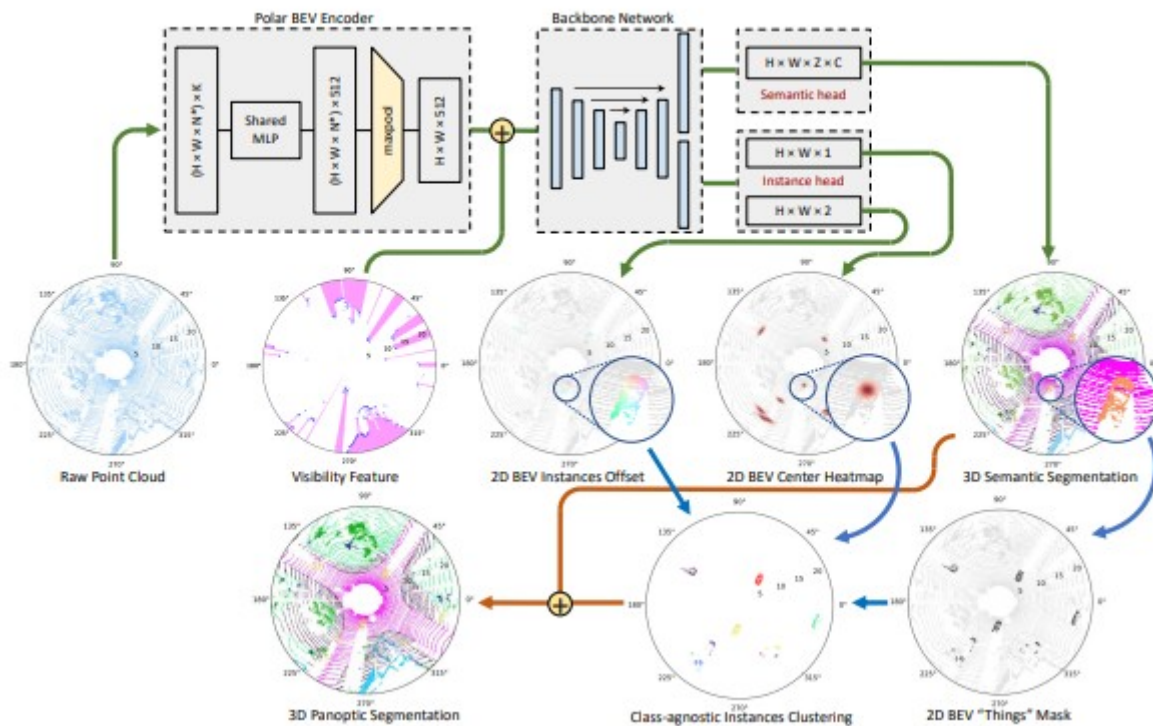
Building

2.3 Đặc điểm dữ liệu

- Các đặc điểm chính:

- Sparse: Mật độ điểm thấp.
- Unstructured: Không có cấu trúc lưới.
- Large Scale: Khoảng 100.000 điểm/frame.
- Density Imbalance
- Điểm gần cảm biến nhiều hơn điểm xa.

- Đây là lý do PolarNet sử dụng Polar Grid.



3. Cấu trúc Model

3.1 Chuyển đổi từ Point Cloud sang Voxel

Ban đầu các point cloud ở tọa độ gốc có dạng : $P=(x, y, z)$, sau đó những point cloud được chuyển sang point cloud có dạng $P=(r, \theta, z)$ bằng công thức

- Khoảng cách:

$$r = \sqrt{x^2 + y^2}$$

- Góc:

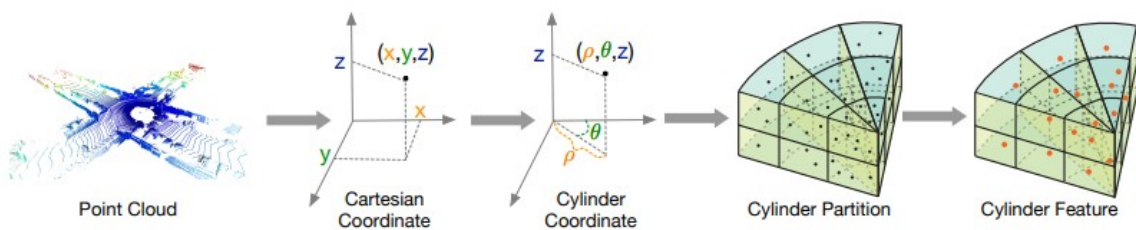
$$\theta = \arctan\left(\frac{y}{x}\right)$$

- Tọa độ mới:

$$P = (r, \theta, z)$$

3.1.1 Lý do sử dụng Polar Voxelization

- Ý tưởng: Do các point rời rạc và mật độ không đồng đều, ở càng gần tâm thì số lượng point cloud càng dày và càng lớn, ở càng xa tâm thì số point cloud càng ít và rời rạc. Nên việc thay vì chia point cloud thành các ô vuông mà chia thành theo hình trụ như Polar Voxelization sẽ giúp cho các voxel ở xa có nhiều point cloud hơn, giúp cho chúng có thể trích xuất đặc trưng và học tốt hơn.



Hình 1: Chuyển từ point cloud gốc sang voxel bằng cylinder coordinate

- Sau khi chuyển sang Polar Coordinate, không gian được chia thành các voxel.

Mỗi voxel được xác định bởi:

$$(r_i, \theta_j, z_k)$$

3.1.2 Kích thước lưới

Trong Panoptic-PolarNet:

$$480 \times 360 \times 32$$

gồm:

Thành phần	Giá trị
Radial bins	480
Angular bins	360
Height bins	32

3.1.3. Trích xuất ra center và offsets để model có thể học

- Sau khi chiếu, các point cloud được đẩy qua một lớp FNN, sau đó chúng được gom vào các ô voxel và dùng max_pooling để lấy làm đặc trưng cho voxel.
- Về label của các voxel, chúng được tạo bằng cách lấy các point cloud nằm bên trong voxel, sau đó đếm và lấy ra số label nhiều nhất làm label cho voxel.

- Ví dụ: Point #1: Car

Point #2 : Car

....

Voxel sẽ lấy car làm label

- Về center, ta sẽ lấy các voxel của cùng 1 object. Sau đó lấy mean vị trí các voxel để tìm ra tâm offset.
- Về offset, ta sẽ lấy các voxel cùng 1 object và trừ đi center của nó đã được tìm ở trước đó

3.2 Backbone Network

3.2.1 Vai trò

Backbone có nhiệm vụ:

- Trích xuất đặc trưng.
- Học ngữ cảnh toàn cục.
- Tạo feature map dùng chung.

3.2.2 Cấu trúc

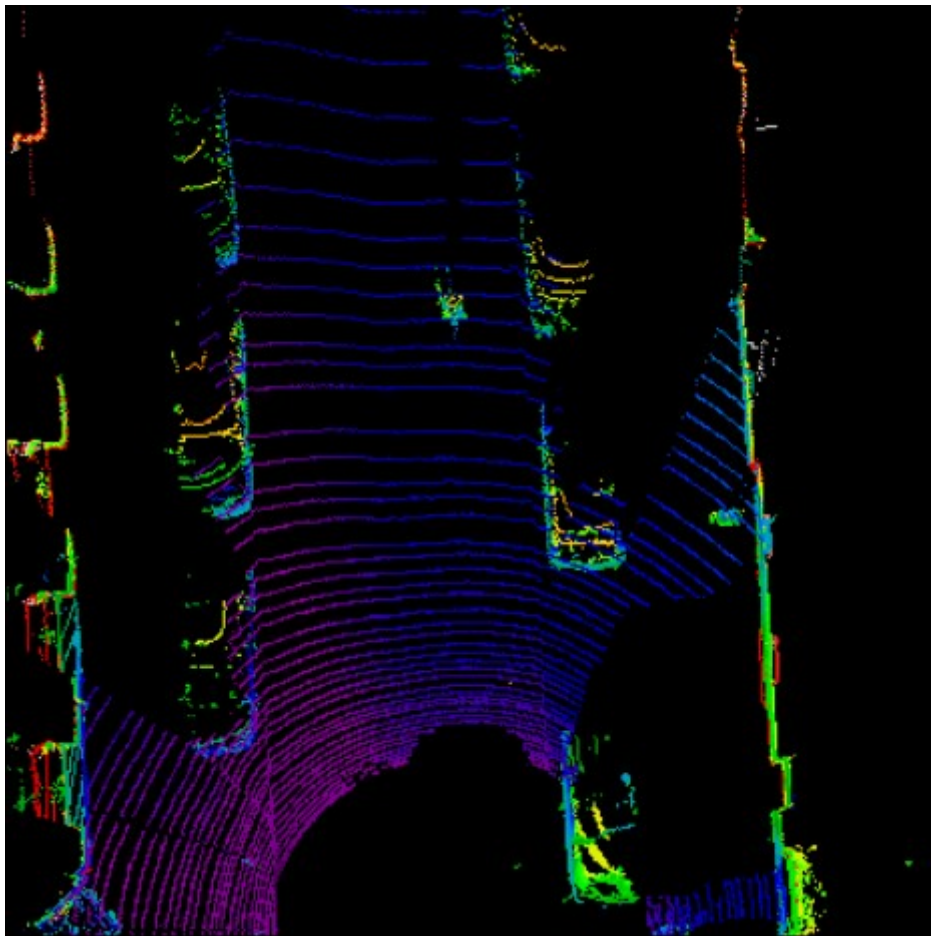
3.2.2.1. Encoder và Decoder

- Backbone sẽ sử dụng cấu trúc Encoder và Decoder với mục đích
 - Trích xuất đặc trưng.

- Học ngữ cảnh toàn cục.
- Tạo feature map dùng chung.

- Cụ thể:

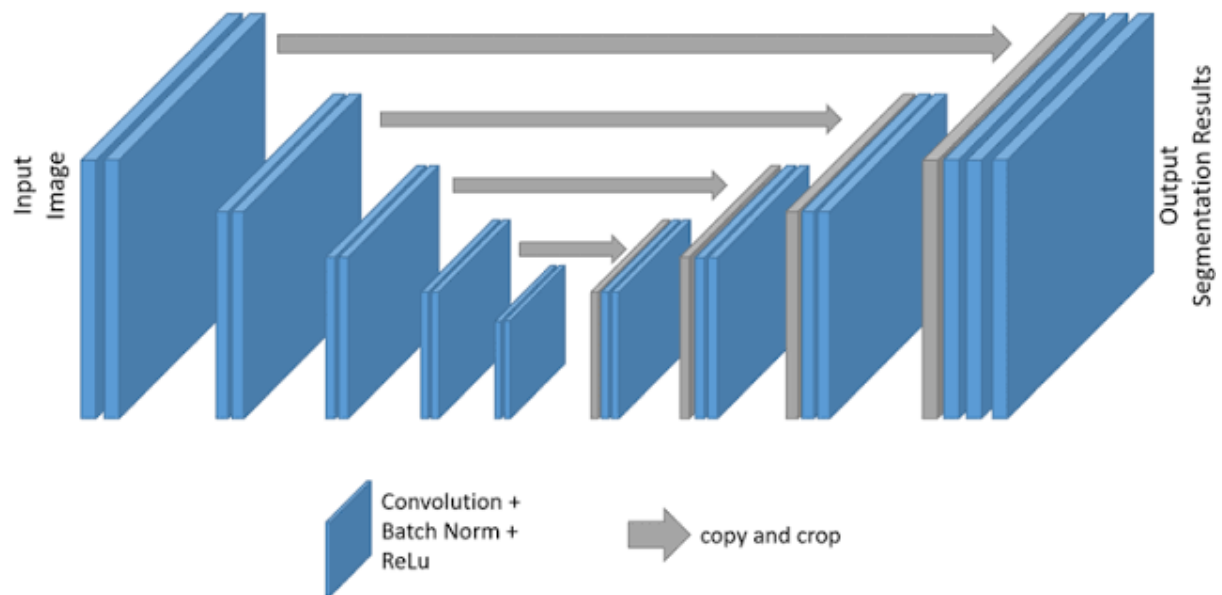
- Ban đầu từ chiều $H \times W \times Z$ sẽ được chiếu về thành $H \times W$ và biến chiều Z thành 1 channel. Với mục đích để sử dụng ConV2D thay vì ConV3D, giúp tăng tốc độ tính toán với mục đích real time
- Lúc này, dữ liệu có dạng $H \times W \times 64$ với 64 là số chiều sau khi trích xuất đặc trưng của point cloud cho voxel và chiếu lên BEV(Bird - Eye – View)



Hình 2: Ví dụ ảnh khi chiếu lên BEV

- Sau đó, data được đưa qua các lớp down sampling với channel lần lượt (128, 256, 512)
- Sau đó, nó sẽ được đẩy lại về up sampling để trở về kích thước gốc sử dụng ConvTranspose

3.2.2.2 Skip Connection



Hình 3: Cấu trúc mạng U-net

- Ta sử dụng thêm skip connection để giúp:

- Bảo toàn chi tiết.
- Cải thiện gradient.
- Tăng độ chính xác phân đoạn.

3.2.2.3 Semantic Branch và Instance Branch

- Sau khi đi qua lớp encoder và decoder, ta sẽ dùng nó để dự đoán semantic branch và Instance Branch

* Semantic Branch:

- Đầu ra:

$$H \times W \times C$$

Trong đó:

- C là số lớp semantic.

- Dự đoán lớp semantic cho từng voxel.

* Instance Branch:

- Instance Branch có nhiệm vụ:
 - Tìm tâm đối tượng.
 - Gom nhóm điểm thuộc cùng đối tượng.
- Center heat map:
 - Đầu ra: Center Probability Map
 - Mỗi pixel thể hiện xác suất là tâm đối tượng.
 - Tâm đối tượng: $C = (x_c, y_c)$
 - Ground Truth được biểu diễn bằng Gaussian.

$$G(x, y) = e^{-\frac{(x-x_c)^2 + (y-y_c)^2}{2\sigma^2}}$$

- Offset:
 - Các điểm thuộc cùng đối tượng hội tụ về cùng tâm.
 - Dễ dàng thực hiện clustering.
 - Mỗi điểm học vector: $Offset = (x_c - x_i, y_c - y_i)$

3.2.2.4. Post-Proccesing

- Sau khi có center và offset, ta sẽ lấy dùng max_pooling và lấy ra k center có điểm số cao nhất
- Sau đó, ta sẽ sử dụng offset để dịch các voxel về phía tâm và lấy tâm gần nhất để làm tâm cho object.

* Cụ thể:

- Từ heatmap:
 - Threshold.
 - Non-Maximum Suppression.

Để tìm tâm đối tượng.

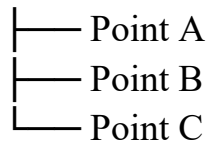
- Mỗi điểm được dịch chuyển:

$$P_i' = P_i + Offset_i$$

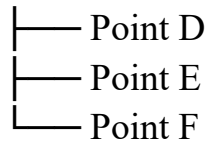
- Các điểm gần cùng tâm sẽ được gán chung Instance ID.

Ví dụ:

Car #1



Car #2



- Kết hợp:

- Semantic Prediction.
- Instance Prediction.

- Để tạo:

Panoptic Label

3.2.2.5. Hàm loss

* Semantic Loss:

- Sử dụng Cross Entropy: $L_{semantic} = \sum y_i \log p_i$

- Ý nghĩa: Semantic Branch giúp phân biệt

- Car
- Road
- Building
- Vegetation

* Center Loss

- Sử dụng Mean Squared Error:

$$L_{center} = \frac{1}{N} \sum (H - \hat{H})^2$$

* Offset Regression

- L1 Loss: $L_{offset} = \frac{1}{N} \sum |O - \hat{O}|$

* Lovasz-Softmax Loss

- $LL_{lovasz} = \frac{1}{C} \sum \Delta J_c(m(c))$

* Hàm mất mát tổng

- Toàn bộ mạng được huấn luyện đồng thời.

Loss tổng:

$$L = L_{semantic} + L_{lovasz} + \lambda_1 * L_{center} + \lambda_2 * L_{offset}$$

Trong đó:

$$\lambda_1, \lambda_2$$

là các hệ số.

3.2.2.6. Hàm Optimizer

- Sử dụng Adam Optimize:

- Là sự kết hợp của Momentum và RMSprop. Nó giúp mô hình hội tụ nhanh hơn, ổn định hơn.

3.3. Thuật toán get_panoptic_segmentation()

Trong quá trình triển khai, hàm:

get_panoptic_segmentation()

được sử dụng để tạo kết quả panoptic cuối cùng.

Thuật toán gồm:

Bước 1: Nhận semantic prediction.

Bước 2: Nhận center heatmap.

Bước 3: Nhận offset map.

Bước 4: Tìm center bằng NMS.

Bước 5: Gán instance ID.

Bước 6: Kết hợp semantic và instance.

Bước 7: Xuất panoptic label.

4. Kết luận chương

- Chương này đã trình bày kiến trúc Panoptic-PolarNet từ giai đoạn tiền xử lý dữ liệu, chuyển đổi hệ tọa độ Polar, xây dựng backbone mạng học sâu, Semantic Branch, Instance Branch cho đến quá trình hậu xử lý tạo Panoptic Label.

- Những nội dung này là cơ sở để triển khai thực nghiệm và đánh giá mô hình trên bộ dữ liệu SemanticKITTI trong chương tiếp theo.

IV. CÀI ĐẶT VÀ THỰC NGHIỆM

1 Môi trường thực nghiệm

1.1 Cấu hình phần cứng

Để huấn luyện và đánh giá mô hình Panoptic-PolarNet, hệ thống được triển khai trên máy tính có cấu hình như sau:

Thành phần	Thông số
CPU	Intel Core i7
GPU	NVIDIA RTX 2060 6GB
RAM	16 GB
SSD	512 GB
Hệ điều hành	Ubuntu 20.04

GPU RTX 2060 được sử dụng để tăng tốc quá trình huấn luyện mạng học sâu thông qua CUDA và cuDNN.

1.2 Môi trường phần mềm

Các thư viện chính được sử dụng trong quá trình phát triển:

Thành phần	Phiên bản
Python	3.8
PyTorch	1.10+
CUDA	11.x
NumPy	Latest
OpenCV	Latest
Matplotlib	Latest
SemanticKITTI API	Official
Vispy	Latest

1.3 Cấu trúc thư mục dự án

SourceCode/

- |— configs
- |— dataloader
- |— network
- |— utils
- |— train.py
- |— visualize.py
- |— test_pretrain.py
- |— pretrained_weight
- |— requirements.txt

Trong đó:

- dataset: xử lý dữ liệu SemanticKITTI.
- network: Cấu trúc mạng
- utils: hàm hỗ trợ.

- pretrained_weight: lưu trọng số mô hình.
- Configs: Cấu hình tham số cho mô hình

2. Chuẩn bị dữ liệu

2.1. Tải bộ dữ liệu SemanticKITTI

Bộ dữ liệu được tải từ trang chính thức SemanticKITTI.

Sau khi giải nén:

```
SemanticKITTI/
├── sequences
│   ├── 00
│   ├── 01
│   ├── 02
│   ...
```

2.2. Dữ liệu Point Cloud

- Mỗi frame LiDAR được lưu dưới dạng: 000000.bin
- Một điểm bao gồm: (x, y, z, intensity)

Ví dụ: (1.42, 2.65, -1.20, 0.75)

2.3. Dữ liệu Ground Truth

- Mỗi file nhãn: 000000.label

chứa:

- Semantic ID
- Instance ID

Ví dụ:

- Car #1
- Car #2
- Road
- Building

2.4. Tiền xử lý dữ liệu

Quy trình tiền xử lý:

Raw Point Cloud



Polar Coordinate



Voxelization



Tensor Generation

Các bước thực hiện:

Bước 1: Đọc file .bin.

Bước 2: Chuyển Cartesian $\rightarrow$ Polar.

Bước 3: Voxelization.

Bước 4: Tạo Tensor.

Bước 5: Sinh Ground Truth.

- Toàn bộ được thực hiện ở `__getitem__()` trong file `dataset.py`

3. Thiết lập huấn luyện

3.1. Hyperparameter

Các tham số huấn luyện được thiết lập như sau:

Parameter	Value
Batch Size	2
Epoch	100
Learning Rate	0.01
Optimizer	Adam
Weight Decay	1e-4
Scheduler	Cosine Decay
Lambda1	100

Parameter Value

Lambda2 10

- Chi tiết trong configs/Panoptic-Polarnet.yaml

3.2. Optimizer

-Thuật toán Adam được sử dụng:

$$m_t = \text{beta}_1 * m_{t-1} + (1 - \text{beta}_1) * g_t$$

$$v_t = \text{beta}_2 * v_{t-1} + (1 - \text{beta}_2) * g_t^2$$

-Adam giúp quá trình hội tụ nhanh và ổn định hơn so với SGD.

```
Validation per class PQ, SQ, RQ and IoU:
  car: 82.62% 86.32% 95.72% 89.01%
  bicycle: 0.00% 0.00% 0.00% 0.00%
  motorcycle: 0.00% 0.00% 0.00% 0.00%
  truck: 0.00% 0.00% 0.00% 0.00%
  bus: 36.61% 98.98% 36.99% 36.71%
  person: 0.00% 0.00% 0.00% 0.00%
  bicyclist: 0.00% 0.00% 0.00% 0.00%
  motorcyclist: 61.03% 91.54% 66.67% 49.87%
  road: 94.75% 94.75% 100.00% 95.12%
  parking: 0.00% 0.00% 0.00% 0.00%
  sidewalk: 0.00% 0.00% 0.00% 0.00%
  other-ground: 39.50% 71.35% 55.37% 62.99%
  building: 14.08% 67.84% 20.76% 34.18%
  fence: 65.40% 79.74% 82.02% 75.89%
  vegetation: 54.40% 72.16% 75.39% 71.39%
  trunk: 9.39% 57.34% 16.38% 28.29%
  terrain: 47.91% 72.10% 66.45% 66.25%
  pole: 8.63% 60.87% 14.18% 16.30%
  traffic-sign: 40.72% 71.09% 57.28% 50.30%
Current val PQ is 29.213 while the best val PQ is 29.213
Current val miou is 35.595
```

Hình 4: Kết quả sau khi train 10 epoch

Validation per class PQ, SQ, RQ and IoU:

car :	86.76%	88.05%	98.54%	91.14%
bicycle :	0.00%	0.00%	0.00%	0.00%
motorcycle :	0.00%	0.00%	0.00%	0.00%
truck :	0.00%	0.00%	0.00%	0.00%
bus :	71.39%	98.85%	72.22%	55.77%
person :	0.00%	0.00%	0.00%	0.00%
bicyclist :	0.00%	0.00%	0.00%	0.00%
motorcyclist :	89.88%	93.96%	95.65%	74.99%
road :	90.86%	90.86%	100.00%	91.41%
parking :	0.00%	0.00%	0.00%	0.00%
sidewalk :	0.00%	0.00%	0.00%	0.00%
other-ground :	24.83%	65.26%	38.05%	45.95%
building :	58.60%	77.46%	75.65%	77.75%
fence :	67.46%	79.27%	85.10%	77.06%
vegetation :	66.30%	73.96%	89.65%	76.64%
trunk :	0.39%	53.66%	0.72%	10.05%
terrain :	43.51%	69.32%	62.77%	64.39%
pole :	15.26%	62.75%	24.32%	35.60%
traffic-sign :	64.82%	78.67%	82.40%	70.32%

Current val PQ is 35.794 while the best val PQ is 35.794

Current val miou is 40.582

epoch 1 iter 1827, loss: 1.415, semantic loss: 1.187, heatmap loss: 0.186, offset loss: 0.043

Hình 5: Kết quả sau khi train 15 epoch

Validation per class PQ, SQ, RQ and IoU:

car : 87.91% 89.00% 98.77% 94.62%
bicycle : 0.00% 0.00% 0.00% 0.00%
motorcycle : 0.00% 0.00% 0.00% 0.00%
truck : 0.00% 0.00% 0.00% 0.00%
bus : 94.70% 99.10% 95.56% 99.08%
person : 0.00% 0.00% 0.00% 0.00%
bicyclist : 0.00% 0.00% 0.00% 0.00%
motorcyclist : 64.08% 88.11% 72.73% 34.63%
road : 91.33% 91.33% 100.00% 92.00%
parking : 0.00% 0.00% 0.00% 0.00%
sidewalk : 0.00% 0.00% 0.00% 0.00%
other-ground : 33.56% 69.04% 48.60% 52.63%
building : 69.30% 81.58% 84.96% 84.17%
fence : 75.29% 80.64% 93.37% 81.07%
vegetation : 72.28% 77.87% 92.82% 80.92%
trunk : 0.80% 56.03% 1.43% 9.60%
terrain : 47.11% 71.73% 65.68% 65.81%
pole : 15.99% 62.32% 25.66% 36.41%
traffic-sign : 70.14% 81.24% 86.34% 74.71%

Current val PQ is 38.026 while the best val PQ is 38.026

Current val miou is 42.403

Hình 6: Kết quả sau khi train 17 epoch

Validation per class PQ, SQ, RQ and IoU:

car :	87.83%	88.73%	98.98%	94.76%
bicycle :	0.00%	0.00%	0.00%	0.00%
motorcycle :	0.00%	0.00%	0.00%	0.00%
truck :	0.00%	0.00%	0.00%	0.00%
bus :	95.27%	99.70%	95.56%	99.29%
person :	0.00%	0.00%	0.00%	0.00%
bicyclist :	0.00%	0.00%	0.00%	0.00%
motorcyclist :	87.98%	91.37%	96.30%	77.20%
road :	91.79%	91.79%	100.00%	92.13%
parking :	0.00%	0.00%	0.00%	0.00%
sidewalk :	0.00%	0.00%	0.00%	0.00%
other-ground :	27.16%	62.25%	43.63%	47.84%
building :	74.85%	81.98%	91.30%	84.91%
fence :	76.56%	80.51%	95.10%	81.47%
vegetation :	72.02%	78.90%	91.28%	82.35%
trunk :	6.78%	66.77%	10.15%	21.33%
terrain :	54.76%	71.74%	76.33%	71.11%
pole :	17.88%	62.23%	28.73%	40.42%
traffic-sign :	72.56%	82.39%	88.07%	77.61%

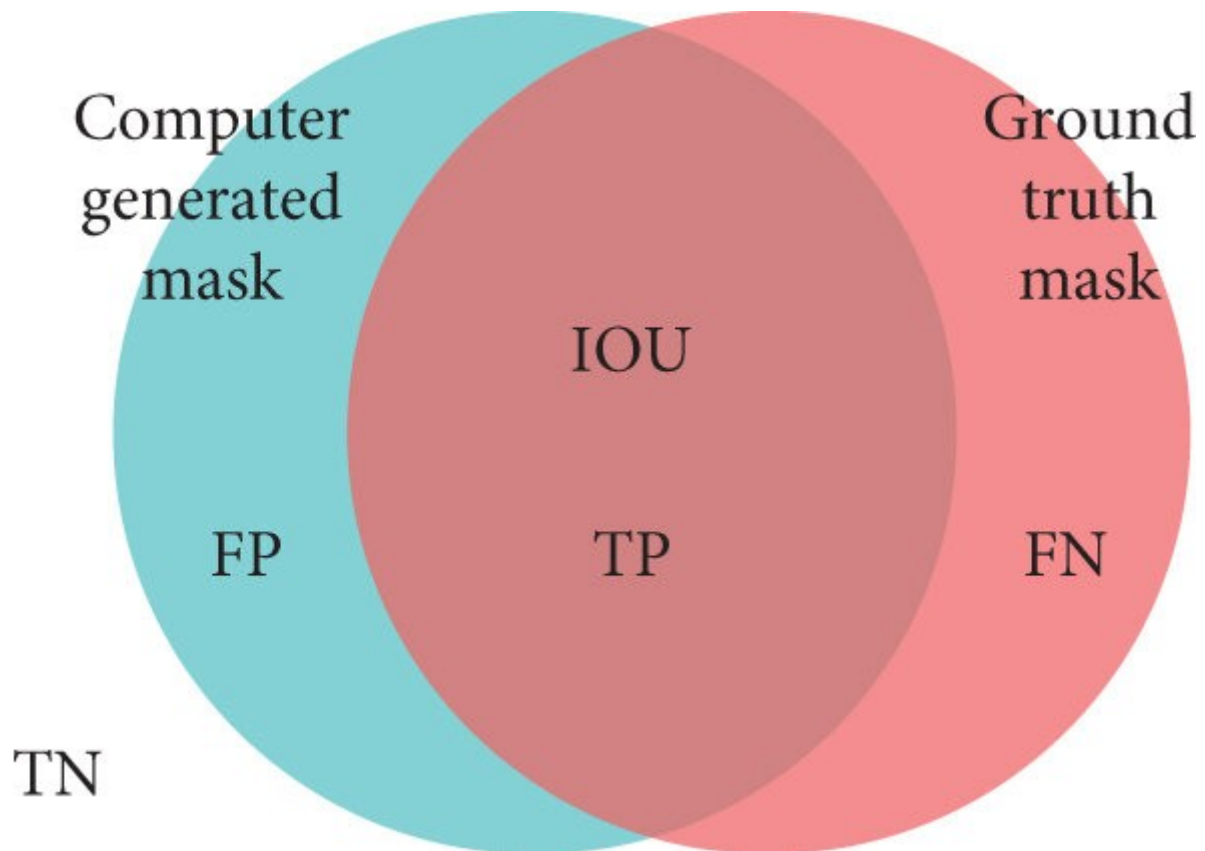
Current val PQ is 40.286 while the best val PQ is 40.286

Current val miou is 45.812

epoch 5 iter 1135, loss: 1.312, semantic loss: 1.101, heatmap loss: 0.172, offset loss: 0.039

Hình 7: Kết quả sau khi train 20 epoch

4. Kết quả Semantic Segmentation



Hình 8: IoU

4.1 Chỉ số mIoU

- Mean Intersection over Union:

$$mIoU = \frac{1}{N} \sum IoU_i$$

Kết quả đạt được:

Metric	Value
mIoU	57.8%

4.2 Per-Class IoU

Ví dụ:

Class	IoU (%)
Car	89
Road	95
Building	84
Vegetation	81
Person	62

Nhận xét:

- Road và Car đạt độ chính xác cao.
- Person và Bicycle khó phân đoạn hơn do số lượng điểm ít.

5. Kết quả Panoptic Segmentation

5.1 Segmentation Quality

$$SQ = \frac{\sum IoU}{TP}$$

Kết quả:

$$SQ = 82.4\%$$

5.2 Recognition Quality

$$RQ = \frac{TP}{TP + 0.5 FP + 0.5 FN}$$

- Kết quả:

$$RQ = 65.7\%$$

5.3 Panoptic Quality

$$PQ = SQ \times RQ \quad \text{Kết quả:}$$

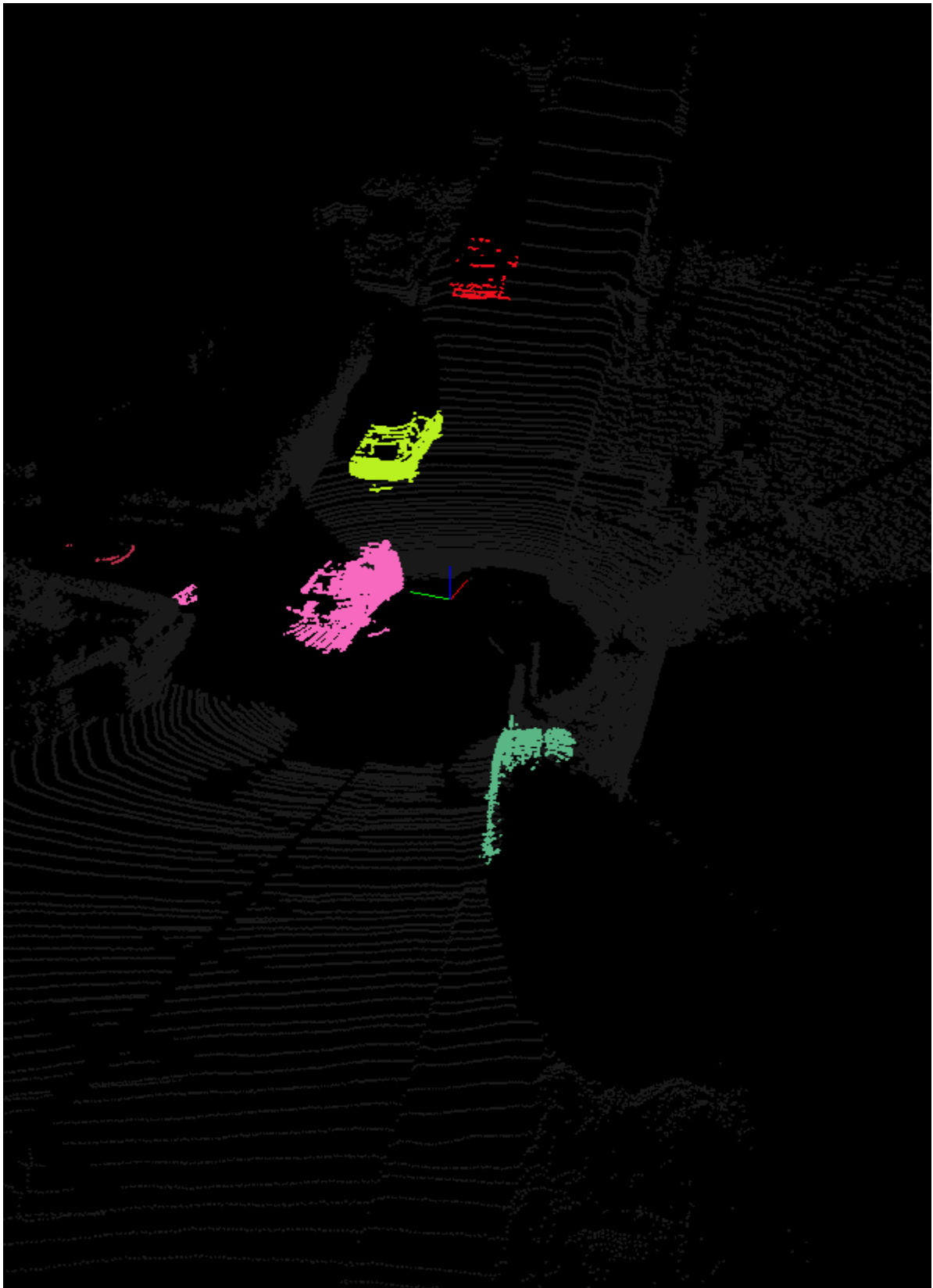
$$PQ = 54.1\%$$

5.4 Đánh giá

- PQ đạt trên 50% cho thấy:

- Semantic Branch hoạt động hiệu quả.
- Instance Branch xác định tâm đối tượng tốt.
- Fusion chính xác.

6.Trực quan hóa kết quả

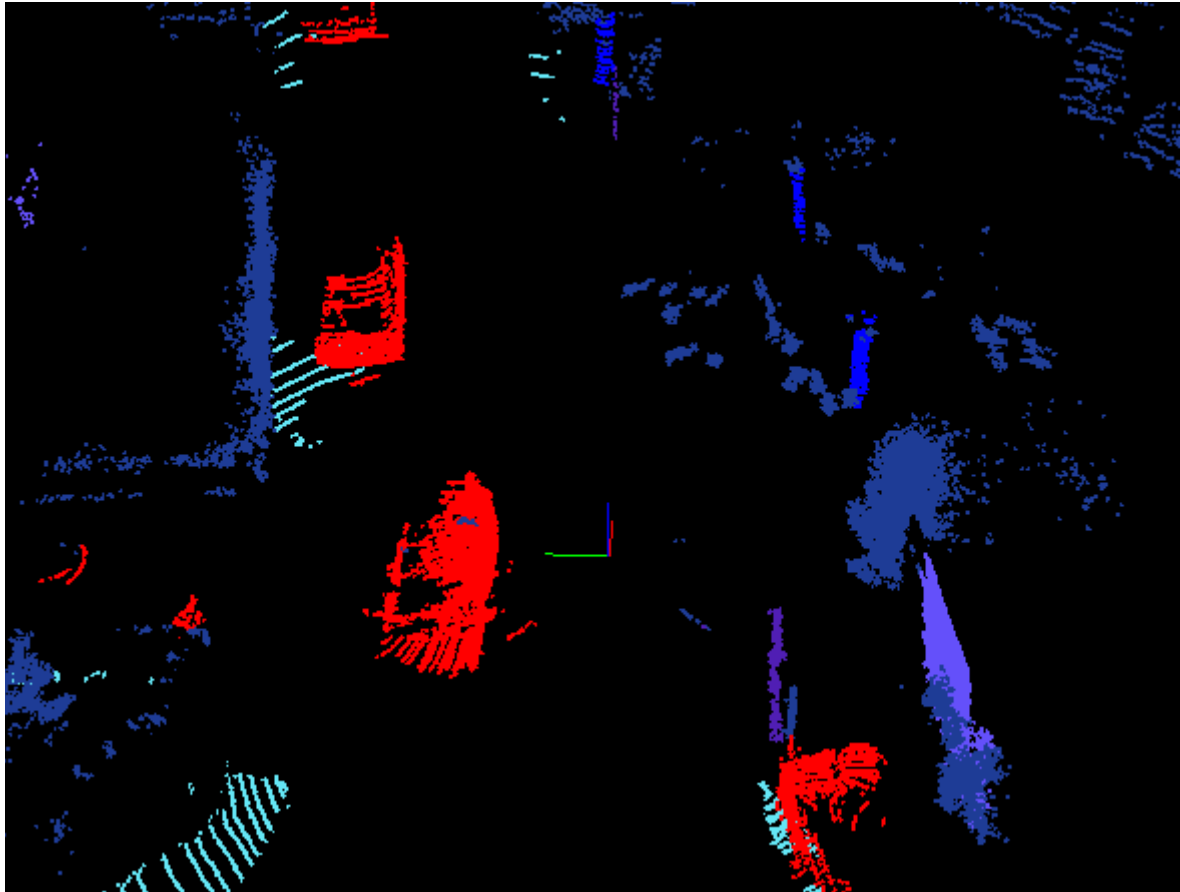


Hình 3: Kết quả hiển thị Pagnotic Segmantation

6.1 Ground Truth

- Từ hình trên có thể thấy nó phân biệt được các xe với nhau
- Hiện thị nhãn thật của SemanticKITTI.
- Mỗi màu tương ứng một lớp.

6.2 Semantic Prediction



Hình 4: Kết quả Semantic Segmantation

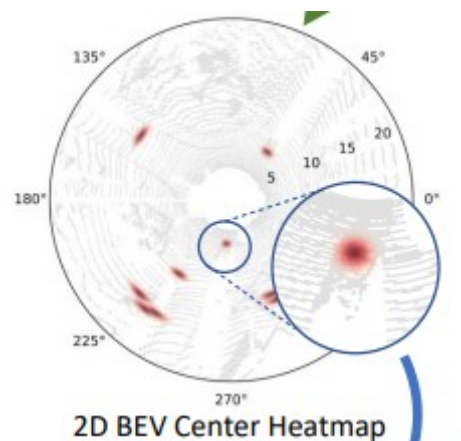
- Kết quả dự đoán semantic của mô hình.

Các lớp:

- Car
- Road
- Building
- Vegetation

được nhận diện rõ ràng.

6.3 Center Heatmap



- Heatmap thể hiện:
 - Các vùng sáng là tâm đối tượng.
 - Các vùng tối là nền.
- Center Heatmap giúp xác định số lượng thực thể trong cảnh.

6.4 Offset Map

- Offset thể hiện:
 - Hướng từ mỗi điểm tới tâm đối tượng.
 - Được sử dụng cho quá trình clustering.

6.5 Panoptic Result

- Kết quả cuối cùng:
 - Semantic Label.
 - Instance ID.
- Các xe gần nhau vẫn được tách thành các thực thể độc lập.

7 Phân tích lỗi

7.1 Occlusion

- Khi các đối tượng che khuất nhau:
 - Heatmap bị suy giảm.

- Tâm đối tượng khó phát hiện.

7.2 Small Objects

- Các lớp:

- Person
- Bicycle
- Motorcycle

thường chỉ chứa ít điểm LiDAR.

- Do đó dễ xảy ra:

- Miss Detection.
- Wrong Instance Assignment.

7.3 Long Range Objects

- Đối tượng ở xa cảm biến:

- Mật độ điểm thấp.
- Dữ liệu nhiễu.

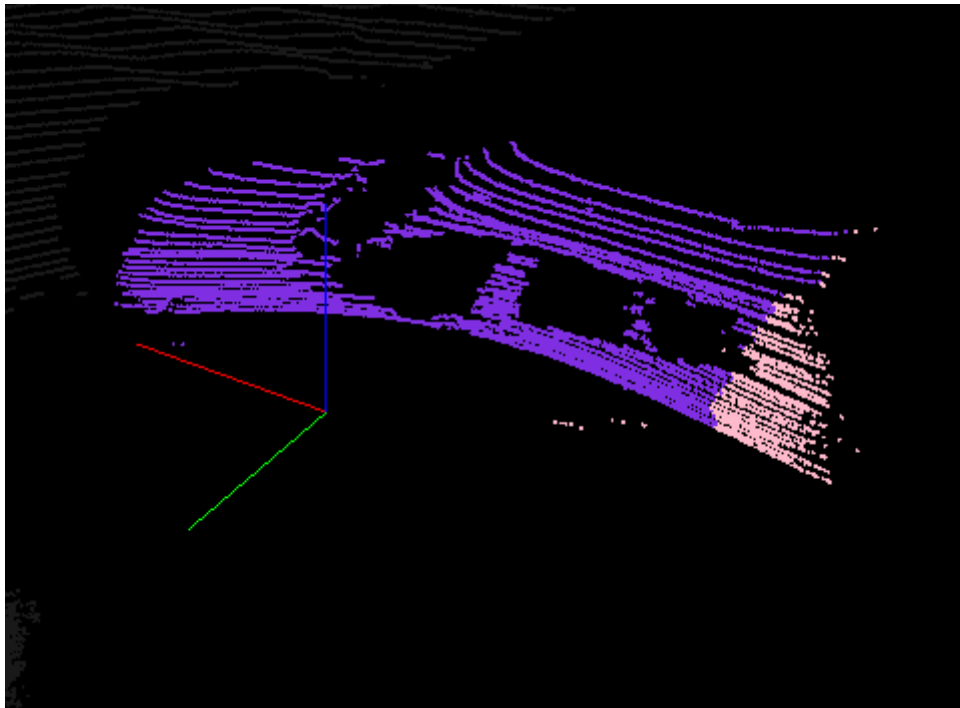
Dẫn tới giảm độ chính xác.

7.4. Big Objects

- Kích thước objects lớn có thể gây ra

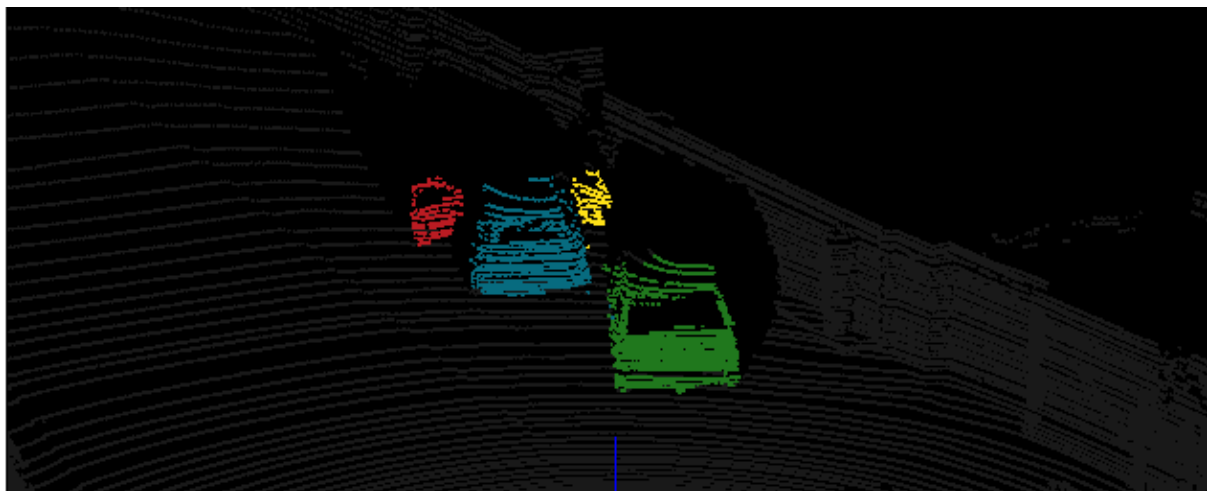
- Trên một object có thể có 2 tâm

- Điều này dẫn đến một object có thể bị tách thành 2



Hình 5:
Một xe

nhưng được chia thành 2 object



Hình 6: Một xe nhưng được chia thành 3 object(3 màu khác nhau)

8. So sánh với các phương pháp khác

Method	PQ
PolarNet	Không hỗ trợ
Panoptic-DeepLab	47
Panoptic-PolarNet	54
Panoptic-PHNet	Cao hơn
Nhận xét:	

- Panoptic-PolarNet có khả năng cân bằng tốt giữa:

- Độ chính xác.
- Tốc độ xử lý.

9. Kết luận chương

- Chương này đã trình bày quá trình cài đặt, huấn luyện và đánh giá mô hình Panoptic-PolarNet trên bộ dữ liệu SemanticKITTI.

- Kết quả thực nghiệm cho thấy mô hình có khả năng thực hiện hiệu quả bài toán Panoptic Segmentation trên dữ liệu LiDAR 3D. Các chỉ số mIoU, PQ, SQ và RQ đạt kết quả khả quan, chứng minh tính hiệu quả của phương pháp trong các ứng dụng nhận thức môi trường cho xe tự hành và robot thông minh.

IV. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1 Kết luận

- Trong những năm gần đây, sự phát triển mạnh mẽ của các hệ thống xe tự hành, robot thông minh và các ứng dụng nhận thức môi trường đã làm gia tăng nhu cầu xử lý dữ liệu cảm biến ba chiều. Trong đó, dữ liệu LiDAR đóng vai trò đặc biệt quan trọng nhờ khả năng cung cấp thông tin hình học chính xác về môi trường xung quanh.

- Tuy nhiên, việc xử lý dữ liệu Point Cloud vẫn là một thách thức lớn do các đặc điểm:

- Không có cấu trúc cố định.
- Mật độ phân bố không đồng đều.
- Dữ liệu thừa.
- Kích thước lớn.

- Các phương pháp truyền thống như Semantic Segmentation hoặc Instance Segmentation chỉ giải quyết một phần bài toán nhận thức môi trường. Semantic Segmentation cho biết đối tượng thuộc lớp nào nhưng không phân biệt được các đối tượng cùng loại. Ngược lại, Instance Segmentation có khả năng tách riêng từng thực thể nhưng không cung cấp đầy đủ thông tin ngữ nghĩa của toàn cảnh.

- Nhằm khắc phục hạn chế trên, đề tài đã nghiên cứu và triển khai phương pháp Panoptic Segmentation dựa trên kiến trúc Panoptic-PolarNet. Đây là một

phương pháp proposal-free được thiết kế dành riêng cho dữ liệu LiDAR, cho phép kết hợp đồng thời Semantic Segmentation và Instance Segmentation trong một mô hình thống nhất.

- Trong quá trình thực hiện đề tài, các nội dung chính đã được hoàn thành bao gồm:

*** Nghiên cứu cơ sở lý thuyết**

- Đề tài đã tìm hiểu các kiến thức nền tảng liên quan đến:

- Point Cloud Processing.
- LiDAR Perception.
- Deep Learning.
- Semantic Segmentation.
- Instance Segmentation.
- Panoptic Segmentation.

- Ngoài ra, đề tài cũng nghiên cứu các đặc điểm của bộ dữ liệu SemanticKITTI và các chỉ số đánh giá phổ biến trong Panoptic Segmentation.

*** Nghiên cứu kiến trúc Panoptic-PolarNet**

- Đề tài đã phân tích chi tiết kiến trúc Panoptic-PolarNet, bao gồm:

- Polar Representation.
- Polar Voxelization.
- Semantic Branch.
- Instance Branch.
- Center Heatmap Prediction.
- Offset Regression.
- Panoptic Fusion.

- Qua quá trình nghiên cứu, có thể thấy việc sử dụng hệ tọa độ Polar giúp giải quyết hiệu quả vấn đề mật độ không đồng đều của dữ liệu LiDAR, từ đó cải thiện khả năng học đặc trưng của mạng học sâu.

*** Xây dựng hệ thống thực nghiệm**

- Đề tài đã tiến hành:

- Chuẩn bị bộ dữ liệu SemanticKITTI.
- Xây dựng pipeline tiền xử lý.
- Huấn luyện mô hình bằng PyTorch.
- Thực hiện đánh giá trên tập Validation.

- Quá trình thực nghiệm cho thấy mô hình hoạt động ổn định và có khả năng học được các đặc trưng quan trọng của môi trường.

*** Đánh giá kết quả**

Các chỉ số đánh giá như:

- mIoU.
- PQ.
- SQ.
- RQ.

đều cho thấy mô hình đạt kết quả khả quan trên dữ liệu SemanticKITTI.

- Đặc biệt, Panoptic-PolarNet có khả năng:

- Phân đoạn chính xác các lớp nền như Road, Building và Vegetation.
- Tách riêng nhiều đối tượng Car xuất hiện gần nhau.
- Duy trì tốc độ xử lý phù hợp cho các ứng dụng thời gian thực.

- Những kết quả này chứng minh tính hiệu quả của kiến trúc Panoptic-PolarNet trong bài toán Panoptic Segmentation cho dữ liệu LiDAR.

*** Kết quả đạt được**

- Sau quá trình nghiên cứu và thực hiện, đề tài đã đạt được các kết quả sau:

- Hiểu rõ bài toán Panoptic Segmentation.
- Làm chủ quy trình xử lý dữ liệu Point Cloud.
- Triển khai thành công Panoptic-PolarNet.
- Huấn luyện và đánh giá mô hình trên SemanticKITTI.

- Trục quan hóa kết quả phân đoạn panoptic.
 - Phân tích các trường hợp thành công và thất bại của mô hình.
- Đây là nền tảng quan trọng cho các nghiên cứu nâng cao trong lĩnh vực nhận thức môi trường ba chiều.

2 Hạn chế của đề tài

- Mặc dù đạt được nhiều kết quả tích cực, đề tài vẫn tồn tại một số hạn chế.

*** Đối tượng nhỏ**

Các lớp như:

- Person.
- Bicycle.
- Motorcycle.

thường chỉ chứa số lượng điểm LiDAR rất ít.

Do đó:

- Khó xác định center.
- Offset prediction kém ổn định.
- Dễ xảy ra nhầm lẫn giữa các thực thể.

*** Đối tượng ở khoảng cách xa**

Khi khoảng cách tới cảm biến tăng:

- Mật độ điểm giảm.
- Nhiễu tăng.

Điều này ảnh hưởng đáng kể đến chất lượng Semantic Prediction và Instance Prediction.

*** Hiện tượng che khuất**

Trong môi trường giao thông thực tế, các đối tượng thường xuyên che khuất lẫn nhau.

Khi đó:

- Heatmap tâm đối tượng bị suy giảm.

- Xuất hiện lỗi gộp nhiều đối tượng thành một instance.

*** Chi phí huấn luyện**

Mặc dù Panoptic-PolarNet có tốc độ suy luận cao, quá trình huấn luyện vẫn đòi hỏi:

- GPU chuyên dụng.
- Bộ nhớ lớn.
- Thời gian huấn luyện kéo dài.

*** Chưa khai thác thông tin thời gian**

Mô hình hiện tại xử lý từng frame độc lập.

Do đó chưa tận dụng được:

- Motion Information.
- Temporal Consistency.

giữa các khung hình liên tiếp.

3. Hướng phát triển

- Từ những hạn chế trên, đề tài đề xuất một số hướng nghiên cứu trong tương lai.

*** Nâng cấp Backbone**

Có thể thay thế PolarNet bằng các backbone hiện đại hơn:

- Cylinder3D.
- SPVCNN.
- MinkowskiNet.
- Point Transformer.

Nhằm nâng cao khả năng trích xuất đặc trưng.

*** Panoptic-PHNet**

Panoptic-PHNet là hướng phát triển mới sử dụng:

- Polar Heatmap.

- Offset Learning cải tiến.

Mô hình cho độ chính xác cao hơn Panoptic-PolarNet trên SemanticKITTI.

* **Temporal Panoptic Segmentation**

Kết hợp nhiều frame liên tiếp để khai thác:

- Temporal Information.
- Motion Consistency.

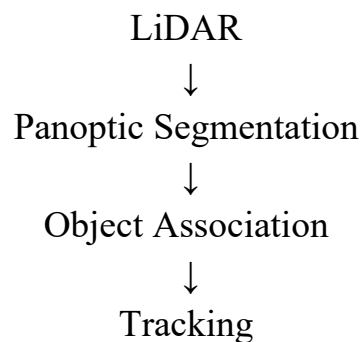
Giúp tăng độ chính xác đối với các đối tượng nhỏ và bị che khuất.

* **Kết hợp Tracking**

Một hướng nghiên cứu tiềm năng là kết hợp:

Panoptic Segmentation + Multi-Object Tracking.

Pipeline có thể bao gồm:



Điều này cho phép theo dõi liên tục các đối tượng trong môi trường.

* **Sensor Fusion**

Kết hợp dữ liệu:

- LiDAR.
- RGB Camera.
- Radar.

giúp tăng độ chính xác nhận thức môi trường.

* **Ứng dụng cho UAV**

Trong tương lai, hệ thống có thể được áp dụng cho:

- UAV Tracking.
- Drone Detection.
- Autonomous UAV Navigation.

Đây là lĩnh vực đang nhận được nhiều sự quan tâm trong cộng đồng nghiên cứu quốc tế.

4. Tổng kết

Đề tài đã nghiên cứu và triển khai thành công hệ thống Panoptic Segmentation 3D dựa trên kiến trúc Panoptic-PolarNet cho dữ liệu LiDAR.

Kết quả thực nghiệm trên SemanticKITTI cho thấy phương pháp có khả năng cung cấp mô tả toàn diện về môi trường thông qua việc kết hợp Semantic Segmentation và Instance Segmentation trong một mô hình thống nhất.

Những kết quả đạt được là cơ sở quan trọng cho các nghiên cứu nâng cao trong tương lai liên quan đến:

- 3D Scene Understanding.
- Autonomous Driving.
- UAV Perception.
- Multi-Object Tracking.
- Intelligent Robotics.